HPS ComplianceVerifier

Automated Monetic Log Verification & Agentic RAG Platform

Comprehensive Technical Onboarding Documentation

Document Specifications:

- Target Audience: Newly Joined AI & QA Engineers
- Author: Senior Software Architect & Technical Writer
- Scope: Frontend, Backend, Parsing Pipeline, RAG/LLM, Infrastructure, Error Handling
- Reference Version: 1.0.0 (Corporate HPS Edition)
- Date: July 2026

Table of Contents

1. Project Overview

The **HPS ComplianceVerifier** is an enterprise-grade agentic QA platform developed specifically for the monetic test engineers at HPS. In the electronic payment domain, verifying that transaction processing flows adhere strictly to business specifications is critical. Normally, QA testers manually read dense, raw monetic trace log files of authorization switches (such as PowerCARD). These logs multiplex transactions running concurrently, making manual parsing highly prone to error and time-consuming.

What Problem it Solves:

- **Log Multiplexing De-entanglement:** Log entries from separate concurrent threads are interleaved. The tool automatically maps each entry to its correct transaction sequence using Session ID grouping.
- **Automated Exception Mapping:** Instead of manually tracing exceptions, the tool automatically matches internal function return codes (like CardInSaf returning NOK) against technical business specifications.
- **Dynamic Audit Generation:** It creates a high-quality audit report in PDF with Corporate HPS layouts, describing step-by-step transaction chronologies, highlighted alert rules, and concrete SQL debugging paths.

High-Level Architecture:

The system is divided into a **React Single Page Application** for user interaction, a **FastAPI** backend, a local **Regex-driven Log Parser**, and a **Multi-Agent LangGraph Workflow**. In addition, the platform features a document ingestion subsystem that parses Excel functional spec sheets (Spec_PowerCARD.xlsx) and registers them into a PostgreSQL vector database equipped with the *pgvector* extension for semantic searching.

Folder Structure:

```
hpsproject/
■■■ backend/ # FastAPI & Agentic Subsystem
■ ■■■ app/ # Main Application Modules
■ ■ ■■■ core/ # Agent definitions & graph logic (agent_graph.py)
■ ■ ■■■ services/ # Domain services (log_parser.py, spec_loader.py, etc.)
■ ■ ■■■ storage/ # Local file uploads, specifications, and reports
■ ■■■ requirements.txt # Python dependencies
■ ■■■ ingest_docs.py # Vector Database RAG Ingestion utility
■ ■■■ .env # API Key & environment configs
■■■ frontend/ # React UI Subsystem
■ ■■■ src/ # React source (App.jsx, App.css, main.jsx)
■ ■■■ package.json # Node dependencies
■ ■■■ vite.config.js # Vite compilation configuration
■■■ docker-compose.yml # Orchestration for PostgreSQL + pgvector DB
```

2. Complete Request Flow

The lifecyle of a compliance request spans frontend execution, backend receipt, log parsing, RAG query extraction, agent reasoning, and PDF generation. Below is the step-by-step processing flow:

Stage 1: Trace Log Upload and Prompt Submission

- **Involved Files:** frontend/src/App.jsx
- **Functions/Components:** App() component, runLogAnalysis() handler.
- **Inputs:** A raw monetic trace file (.TXT) and an optional user prompt.
- **Outputs:** A multipart/form-data POST request.
- **Purpose:** Acquires the user's focus criteria (e.g. general audit vs. specific story extraction) and the raw logs.

Stage 2: API Ingestion and Local Storage

- **Involved Files:** backend/app/main.py
- **Functions/Components:** analyze_logs() route handler.
- **Inputs:** Form inputs (file: UploadFile, user_prompt: str).
- **Outputs:** File written to local disk (e.g. backend/app/storage/BASE1_LCH_2.TRC019.TXT) and initialized AgentState.
- **Purpose:** Persists the file for parsing and initializes the LangGraph graph context.

Stage 3: Demultiplexed Parsing (ParserStoryBuilder)

- **Involved Files:** backend/app/core/agent_graph.py, backend/app/services/log_parser.py
- **Functions/Components:** parser_story_node(), parse_trace_file_for_story().
- **Inputs:** state["file_name"]
- **Outputs:** state["log_data_json"] (serialized suspicious transactions chronology).
- **Purpose:** Scans the log file, demultiplexes interleaved sessions, extracts metadata, correlates outgoing response codes with requests using STAN, and tracks function failures.

Stage 4: Specification Retrieval (RAG Retriever)

- **Involved Files:** backend/app/core/agent_graph.py, backend/app/services/spec_loader.py
- **Functions/Components:** rag_spec_retriever_node(), get_spec_context_for_functions().
- **Inputs:** state["log_data_json"] (extracts failed function names).
- **Outputs:** state["rag_context"] (markdown listing matching rules).
- **Purpose:** Queries the functional specification database to provide context to the LLM regarding what the failed functions were supposed to do under spec rules.

Stage 5: Multi-Agent Reasoning (Compliance Auditor)

- **Involved Files:** backend/app/core/agent_graph.py
- **Functions/Components:** compliance_auditor_node(), ChatGoogleGenerativeAI.
- **Inputs:** state["user_prompt"], state["log_data_json"], state["rag_context"].
- **Outputs:** state["final_response"] (formatted Markdown audit text).
- **Purpose:** Integrates the chronological log facts and the functional specification rules to compile a cohesive audit report targeting the user prompt.

Stage 6: PDF Report Building

- **Involved Files:** backend/app/main.py
- **Functions/Components:** reportlab.platypus.SimpleDocTemplate, build().
- **Inputs:** state["final_response"].
- **Outputs:** File written to backend/app/storage/Rapport_Compliance_HPS.pdf.
- **Purpose:** Converts the LLM markdown response into a formatted, print-ready PDF matching HPS corporate branding.

3. Frontend Architecture (React)

The frontend dashboard is designed as a single-page React application configured with Vite. Key architectural characteristics include:

- **Component Structure:** Located inside frontend/src/App.jsx. It is a single component structure split logically into a navigation sidebar (aside) and a main content viewport (main) that displays panels dynamically based on activeTab.
- **State Management:** Uses React's useState hooks to handle file attachments, loader state, API response content, local diagnostic message buffers, and activeAgent state indicators.
- **Upload Ingestion:** Supports .txt/.TXT files via an HTML file input, converting files to a FormData object before executing POST queries.
- **Active Agent HUD:** To enhance UX, the frontend maps backend agent progression using state variable activeAgent, showing live feedback ('Supervisor', 'LogAgent', 'FINISH') to visualize backend graph progression.
- **Response Rendering:** The AI markdown report is rendered natively in the browser using the react-markdown component, allowing rich text, bullet points, and code block formatting.
- **Error Boundaries & State UI:** Catches axios failures, reading custom error details passed by the FastAPI validation schema and formatting them in a red-tinted alert UI box.

4. Backend Architecture

The backend is built as an asynchronous FastAPI application, serving as the central controller orchestrating file management, database queries, and agentic workflows.

1. API Endpoint Layout

- **GET /:** Basic status check ensuring connectivity.
- **POST /api/v1/logs/analyze:** Main endpoint. Ingests user_prompt and trace files. Triggers LangGraph, builds PDF, and returns transaction chronology data and report markdown text.
- **GET /api/v1/logs/download-pdf:** Downloads the built PDF Rapport_Compliance_HPS.pdf if it exists and exceeds minimum file-size validation.

2. Request Lifecycle

FastAPI processes incoming requests in the following sequence:

- **CORS Validation:** Intercepts query, verifying origin policies to permit react integrations (ports 5173, etc.).
- **Payload Validation:** Form data parameters are validated (user_prompt cannot be empty, files must have a .TXT suffix).
- **Storage Action:** Ingested logs are stored under backend/app/storage/.
- **Graph Invocation:** Executes the LangGraph state machine synchronously.
- **Document Builder:** ReportLab compiles the final response into PDF.
- **Response Marshalling:** The API returns JSON metadata to the frontend. The PDF remains on local storage, ready for download requests.

5. Parsing Pipeline

The parsing pipeline in **log_parser.py** is the core technical component of the application. It translates raw, interleaved ASCII log dumps into clean transaction histories without using an LLM, saving hundreds of thousands of context window tokens.

1. Multi-Thread Demultiplexing by Session ID

PowerCARD processes transactions concurrently. As a result, trace lines belonging to separate requests are interleaved in a single log file. Simple line-by-line reading would corrupt the flow representation (cross-transaction contamination).

To solve this, the parser extracts the unique session ID of every log line using the pattern:

```
RE_SESSION = re.compile(r'^\S+\s+\S+\s+\S+\s+(\S+?)\|')
```

Every line matching a session ID is collected in a session-specific state map: *sessions[session_id]*. A transaction sequence is initialized when 'Start DumpVisa()' is matched and concluded when another 'Start DumpVisa()' appears on the same session or the file ends.

2. Network Response Code Cross-Correlation via STAN

In PowerCARD, the incoming request (MTI 1100) and outgoing response (MTI 1110) are handled on different network sockets, and thus are printed under different Session IDs. The only shared identifier is the **STAN (System Trace Audit Number - FLD 011)**.

To correlate response codes back to request sessions, the parser executes a **Pre-Pass Scan** (*_build_response_code_map*):

- Scans the log file for lines containing MTI 1110.
- Matches the STAN (*FLD (011)*) and the Response Code (*FLD (039)*).
- Saves a mapping { stan_number: response_code }.
- During the main demultiplexing loop, when concluding a transaction, the parser looks up the transaction's STAN in the mapping and appends the response code to the story chronologically.

3. Dynamic Rule Generation from Excel Specifications

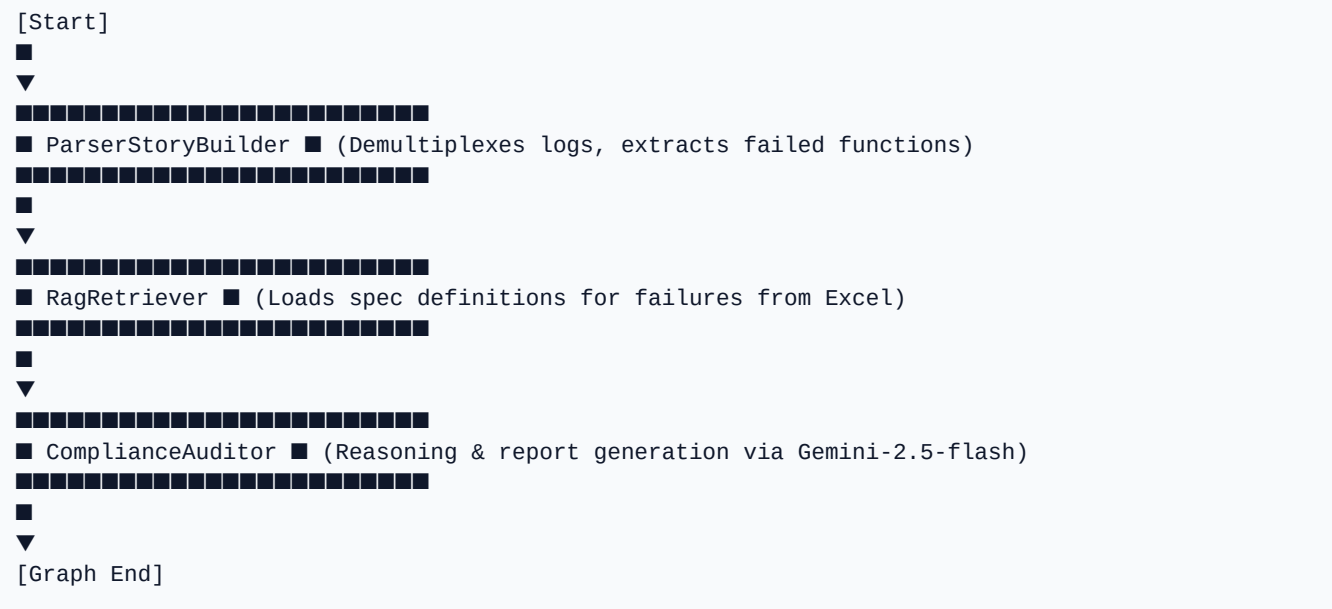
Instead of hardcoding function names like CardInSaf or CheckLimits, the parser calls *get_monitored_function_names()*. It dynamically generates regex search patterns for every function listed in the spreadsheet:

```
rf'{re.escape(func)}\s*\(?\s*(NOK|-1|-2|!=\s*OK)'
```

This means that if a new function is added to *Spec_PowerCARD.xlsx*, the parser will automatically detect its failures in the logs without code changes.

6. AI / LLM Pipeline

The AI orchestration pipeline is built on **LangGraph**, allowing developers to model agent behavior as a state machine. The workflow consists of three consecutive nodes:



Node 1: ParserStoryBuilder
Calls the Python parser to extract the chronologies. To avoid overwhelming the LLM's context window, it filters out healthy transactions and keeps only transactions containing alerts. It serializes this structure to JSON.

Node 2: RagRetriever
Identifies the set of unique failed functions from the log JSON. It queries the local Excel sheet metadata to build a markdown block of specifications describing what those functions are supposed to do under business rules.

Node 3: ComplianceAuditor
Combines the user's specific request prompt, the transaction JSON logs, and the RAG specification context. It compiles the prompt and executes a call to Gemini, which returns the audit report.

7. Embeddings

Embeddings convert textual specifications into high-dimensional numerical vectors. This mathematical representation enables the system to perform semantic searching, mapping natural language questions to concrete function requirements.

- **Embedding Model:** The system uses *models/gemini-embedding-001* from Google via LangChain's GoogleGenerativeAIEmbeddings.
- **Vector Dimensions:** The model generates dense vectors with **768 dimensions**.
- **Semantic Metric:** Embeddings are matched using **Cosine Similarity** (which measures the cosine of the angle between two vectors, ranging from -1 to 1) or L2 distance.
- **Why Embeddings are Necessary:** In monetics, QA engineers rarely search for exact code symbols. They ask questions like: 'How does the system handle credit card limits when offline?' By comparing the embedding of this question with the embeddings of the Excel spec sheet, the retriever locates functions like *CheckLimits* or *CardInSaf*, even if the user's question didn't use those terms.

8. Vector Database

The application uses **PostgreSQL** with the **pgvector** extension as its vector database. Below are the design specifications of the storage engine:

- **Container Configuration:** Defined in docker-compose.yml as a service named 'db' running the *pgvector/pgvector:pg16* image.
- **Schema Design:** Under the hood, pgvector stores embeddings in a table associated with the LangChain collection name: *hps_specifications*. The schema consists of columns for the document content string, the 768-dimensional embedding vector, and metadata JSON.
- **Metadata Attributes:** Stores the origin filename ('Spec_PowerCARD.xlsx') and the target function name (e.g. 'CardInSaf') to allow filtering and exact matches.
- **Indexing:** Re-indexes the database during document ingestion (using *pre_delete_collection=True*) to keep vectors aligned with the source Excel file.
- **Retrieval Process:** Queries execute via LangChain's PGVector similarity search, computing vector distance over the database tables and returning the top-k document chunks.

9. Prompt Engineering

Prompt engineering controls the behavior and output formatting of the LLM. The prompt layout in `agent_graph.py` is designed to prevent hallucinations and enforce strict business formatting:

1. System Prompt Constraints

- Forces the model to adopt the persona of an 'Agent Expert en Audit et Conformité Monétique pour les testeurs d'HPS'.
- Restricts output strictly to what the user asks (e.g. if the user only asks for the story, the LLM must suppress alerts and debugging paths). This reduces token usage and prevents cluttering the UI.

2. Context Injection Schema

The prompt template injects context dynamically using three key variables:

- **user_prompt:** The user's query.
- **log_data_json:** The filtered transaction JSON (containing chronological facts and trace lines).
- **rag_context:** The real specification requirements matching the failed functions.

3. Hallucination Suppression

- The model is instructed to make direct references to the provided RAG context.
- If a function is not present in the RAG context, the model is forbidden from inventing specifications for it.

10. Retrieval Process

The retrieval process locates relevant functional rules when an exception occurs. The system supports two modes of retrieval:

1. Static Functional Mapping (Active)

Currently active in `agent_graph.py`. When the log parser identifies function names returning NOK (e.g. CardInSaf), the retriever node fetches their exact metadata from the Excel sheet via *get_spec_context_for_functions()*. This acts as a direct, deterministic lookup database.

2. Semantic RAG Search (Target Integration)

Implemented in *ingest_docs.py* and mapped in the frontend. When a QA engineer queries the knowledge base directly, the system uses semantic retrieval:

- The user's query is converted to a vector using the Google Generative AI Embeddings model.
- A similarity search is executed against the *hps_specifications* collection in the pgvector database.
- The top-k (closest vector distance) documents are retrieved and returned as the context.

11. LLM Response Generation

Once the prompt is constructed, it is sent to the LLM (Gemini-2.5-flash) to generate the final response. The model processes the input and formats its output as follows:

- **Generative Reasoning:** The model matches the chronological logs against the specification context to determine compliance. For example, if the logs show that *CardInSaf* returned NOK and the specs indicate that this means a card is blacklisted, the model explains that the transaction was declined due to card status.
- **Actionable Diagnostics:** The model suggests debugging steps, such as verifying specific database tables (e.g., the blacklist table) or checking routing configurations.
- **Output Formatting:** The response is structured in clean Markdown, using bold text, bullet points, and code blocks for readability.

12. Error Invalidation & Handling

Robust error handling is implemented across all layers of the application to prevent failures and ensure system stability:

- **Invalid/Missing File Upload:** Checked at the FastAPI entry point. If the uploaded file is empty or does not end in .txt/.TXT, the API returns a 400 Bad Request error.
- **Empty Log Handling:** If the log parser finds no transactions, it returns an empty list, and the retriever node bypasses RAG lookup.
- **API Key Validation:** Checked during backend startup. If no Google API key is found in the environment variables, the system raises a RuntimeError to prevent startup failures during API calls.
- **ReportLab PDF Rendering Safeguard:** ReportLab fails if it encounters invalid HTML tags. The PDF builder escapes the text using `html.escape` and uses regex to convert Markdown markers (like bold or lists) into ReportLab-safe tags. If rendering still fails, a fallback routine generates a plain text PDF to ensure the user always receives a readable file.
- **Database Connection Failures:** Ingest scripts are wrapped in try-except blocks to catch connection errors and output helpful troubleshooting instructions.

13. Configuration Reference

Key configuration parameters are managed using environment variables and standard config files:

1. Environment Variables (.env)

- **GOOGLE_API_KEY / GEMINI_API_KEY:** Credentials for Google Generative AI.
- **LOG_STORAGE_DIR:** Directory where uploaded log files are stored.
- **GEMINI_MODEL_NAME:** LLM model name (defaults to gemini-2.5-flash).
- **POWERCARD_SPEC_PATH:** Path to the Spec_PowerCARD.xlsx spreadsheet.

2. Container Setup (docker-compose.yml)

Configures the PostgreSQL vector database database:

- Image: pgvector/pgvector:pg16
- Port: 5432:5432
- DB name: hps_docs_db
- Credentials: postgres / password

3. Python Dependencies (requirements.txt)

- **fastapi / uvicorn:** Web framework and server.
- **langgraph:** Multi-agent orchestration framework.
- **langchain-google-genai / google-generativeai:** Google Gemini API bindings.
- **reportlab:** Dynamic PDF generation library.
- **pgvector / psycopg2-binary:** PostgreSQL driver and vector extension.
- **openpyxl:** Excel file parsing library.

14. Project Architecture Diagrams

Below are text-based representations of the system architecture diagrams, formatted using standard Mermaid syntax:

1. High-Level Architecture Diagram

```
graph TD
User([User / Tester]) <-->|Uploads log / Prompt| React[React Frontend]
React <-->|REST API / JSON| FastAPI[FastAPI Backend]
subgraph FastAPI Services
LogParser[Log Parser] <-- Reads specs --> Excel[Spec_PowerCARD.xlsx]
LangGraph[LangGraph Agents] <-- Invokes --> Gemini[Gemini LLM]
ReportLab[ReportLab Builder] --> PDF[Rapport_Compliance_HPS.pdf]
end
FastAPI -->|Query / Embeddings| DB[(PostgreSQL + pgvector)]
```

2. Parsing Pipeline Diagram

```
graph LR
RawLog[Raw Log File] --> Split[Split by Session ID]
Split --> Parse[Parse PAN, STAN, TX_ID]
Parse --> Match[Match NOK / Error Patterns]
Match --> Correlate[Correlate Response Codes via STAN]
Correlate --> Filter[Filter to Suspicious Transactions]
Filter --> JSON[Log Chronology JSON]
```

3. RAG Pipeline Diagram

```
graph TD
Excel[Spec_PowerCARD.xlsx] --> Ingest[ingest_docs.py Ingestion]
Ingest --> Embed[Generate Embeddings via Gemini]
Embed --> Store[(PGVector Database)]
Query[User Query] --> MatchEmbed[Generate Query Embedding]
MatchEmbed --> Similarity[Cosine Similarity Search]
Similarity --> Context[Retrieve Context Chunks]
```

15. Sequence Diagram

The sequence diagram below shows the interaction flow between system components:

```
sequenceDiagram
autonumber
actor User
participant React as React Frontend
participant FastAPI as FastAPI Backend
participant Parser as Log Parser Service
participant PGVector as PGVector DB / Excel
participant LLM as Gemini LLM API

User->>React: Upload .TXT Log & Submit Prompt
React->>FastAPI: POST /api/v1/logs/analyze
FastAPI->>FastAPI: Save raw log file
FastAPI->>Parser: parse_trace_file_for_story()
Parser->>Parser: Group log lines by Session ID
Parser->>Parser: Correlate request/response via STAN
Parser->>FastAPI: Return Log Chronology JSON
FastAPI->>PGVector: get_spec_context_for_functions(failed_funcs)
PGVector-->>FastAPI: Return Specification Context (RAG)
FastAPI->>LLM: Send Prompt + Log JSON + RAG Context
LLM-->>FastAPI: Return Markdown Audit Report
FastAPI->>FastAPI: Convert Markdown to Corporate PDF
FastAPI-->>React: Return JSON Response
React-->>User: Display On-Screen Markdown Report
User->>React: Request PDF Download
React->>FastAPI: GET /api/v1/logs/download-pdf
FastAPI-->>User: Return Rapport_Compliance_HPS.pdf
```

16. Component Code Walkthrough

Below is a walkthrough of the key files in the codebase, explaining their purpose, key functions, and dependencies:

1. backend/app/main.py

- **Purpose:** Acts as the entry point for the FastAPI application, handling API routing, CORS configuration, and file uploads.
- **Key Functions:** *analyze_logs()* coordinates the execution of the parsing pipeline, LangGraph workflow, PDF generation, and API responses. *download_pdf()* handles PDF download requests.

2. backend/app/core/agent_graph.py

- **Purpose:** Orchestrates the multi-agent LangGraph workflow.
- **Key Functions:** *parser_story_node()* runs the log parser on the uploaded file and stores the transaction JSON in the agent's state. *rag_spec_retriever_node()* extracts failed functions from the logs and retrieves their specifications from the Excel sheet. *compliance_auditor_node()* constructs the final prompt and queries the LLM.

3. backend/app/services/log_parser.py

- **Purpose:** Parses the raw log files to extract transaction stories and detect failures.
- **Key Functions:** *parse_trace_file_for_story()* groups logs by session ID to demultiplex concurrent transactions, uses *_build_response_code_map()* to correlate network responses via STAN, and dynamically checks for function failures using regex.

4. backend/app/services/spec_loader.py

- **Purpose:** Interface for reading functional specifications from Excel.
- **Key Functions:** *load_function_specs()* reads row data from the 'Lib' sheet of the Excel file and caches the result. *get_spec_context_for_functions()* builds the specification context markdown block for the LLM.

5. backend/ingest_docs.py

- **Purpose:** RAG ingestion script that loads data from Excel, generates embeddings, and saves them to pgvector.
- **Key Functions:** *charger_documents_depuis_excel()* reads Excel rows and converts them to LangChain Document objects. *initialiser_et_ingerer()* generates embeddings via Google Generative AI and saves them to PostgreSQL.

6. frontend/src/App.jsx

- **Purpose:** Frontend interface for uploading files and viewing reports.
- **Key Functions:** *runLogAnalysis()* handles file uploading and sends the POST request to the API. Renders the audit report using *react-markdown*.

17. Glossary of Terms

- **RAG (Retrieval-Augmented Generation):** An AI architecture that retrieves relevant documents from a database and provides them as context to the LLM to improve response accuracy.
- **Embedding:** A dense, high-dimensional vector representation of text that captures semantic meaning.
- **Vector Database:** A database specialized in storing and querying vector embeddings.
- **Cosine Similarity:** A metric used to measure similarity between two vectors.
- **STAN (System Trace Audit Number):** A unique 6-digit number assigned to every electronic payment transaction, used as a correlation key.
- **Session ID:** An identifier used to group log lines belonging to the same thread in concurrent logs.
- **Hallucination:** A phenomenon where an LLM generates factually incorrect or unsupported information.

18. Technical Suggestions

Below are architectural recommendations to improve system performance, security, and maintainability:

- **Transition to Semantic Retrieval:** Fully integrate the pgvector database (currently set up in *ingest_docs.py*) into *agent_graph.py*. Replace the local Excel lookup with database similarity search queries to scale to larger documentation sets.
- **Asynchronous Job Processing:** Long-running log analyses can exceed HTTP timeout limits. Offload analysis tasks to an asynchronous task runner like Celery and use WebSockets or polling to update the frontend state.
- **Enhanced Security:** Implement JWT-based authentication for backend endpoints and sanitize file uploads to prevent path traversal vulnerabilities.
- **Performance Optimization:** Implement streaming LLM responses to the frontend using Server-Sent Events (SSE) or WebSockets to reduce perceived latency.

19. Architectural Conclusion

The **HPS ComplianceVerifier** provides an efficient solution for analyzing complex monetic transaction logs. By combining local, regex-driven parsing with agentic workflows and semantic retrieval, the system demultiplexes raw log files and automatically verifies transaction execution against business specifications.

Implementing the suggestions above, such as fully integrating the pgvector database and adopting asynchronous job processing, will help scale the platform and improve performance, making it a robust tool for monetic QA teams.